

06 — Flutter Architecture, Storage & Security

Implementation target: Flutter Android-first, clean architecture, offline-first MVP

Technology

- Flutter + Dart.
- State management: choose one consistent approach, preferably Riverpod or Bloc. Do not mix multiple state-management systems without a reason.
- Local database: choose a mature local database suitable for offline-first structured data.
- Notifications: Flutter-compatible local notification package.
- Charts/calendar: Flutter packages only when needed; avoid excessive dependencies.
- AI provider is optional and must be abstracted.

Suggested Architecture

- Presentation: screens, widgets, view models/controllers.
- Domain: entities, use cases, calculation services.
- Data: repositories, local data source, optional remote data source.
- Core: date utilities, validation, security, configuration and shared constants.
- Medical content: static reviewed content with source metadata and review date.

Core Entities

- UserProfile
- CycleRecord
- PeriodEntry
- SymptomEntry
- FertilityObservation
- IntimacyEntry (private by default)
- PregnancyRecord
- Appointment
- Reminder
- EducationArticle
- PartnerSharePermission
- AIConversationMetadata

Privacy

- Sensitive reproductive data must be treated as highly private.
- App lock with PIN/biometric.
- Encrypt sensitive local storage where supported.
- Do not log reproductive data to debug logs or analytics.

- Do not include sensitive values in crash reports.
- Partner sharing must be explicit, revocable and field-specific.
- Provide export and permanent deletion in Settings.

Free MVP Rule

- No paid backend is required for the first prototype.
- Use local storage first.
- AI can be disabled behind a feature flag.
- Do not build payments, subscriptions or ads in MVP.
- Architecture should leave a clean path for optional backend sync later.

Project Structure

- lib/core/
- lib/features/onboarding/
- lib/features/cycle/
- lib/features/fertility/
- lib/features/pregnancy/
- lib/features/appointments/
- lib/features/education/
- lib/features/partner/
- lib/features/ai/
- lib/data/
- test/

Engineering Rules

- Write tests for all date calculations.
- Use typed models and validation.
- Keep medical constants centralized and documented.
- Do not hard-code UI strings inside business logic.
- Use localization-ready strings from the beginning.
- Keep the app functional without network access.